

Capacity AR

scope-equipos.md

Scope entre equipos

Documento de acuerdo entre equipos sobre quien hace que dentro del monolito modular Capacity AR.

Contexto

El sistema se divide en 2 equipos que trabajan sobre el mismo backend modular. Una sola API, una sola base de datos, modulos internos. No es microservicios.

Mapa De Alto Nivel

```
EMPRESA + ESTUDIANTES
|
v
+-----+
| EQUIPO 1 |
| Gap + Metricas globales |
+-----+
| GapReport JSON
v
+-----+
| EQUIPO 2 (nosotros) |
| Capacitacion + Tutores |
+-----+
```

Flujo unidireccional. Equipo 1 produce el insumo. Equipo 2 lo consume y ejecuta el proceso pedagogico.

Equipo 1 - Gap + Metricas Globales

Responsabilidades

1. Onboarding de estudiantes - Recibir datos del form de Google (webhook o Apps Script). - Crear el student_profile en la base.
2. Gestion de empresas y ofertas - Form web para que la empresa cargue sus datos. - Form web para crear y editar posiciones con habilidades requeridas.
3. Gap Engine - Recibir student + company + target_role. - Calcular gap por habilidad, status (READY/NEEDS_WORK/MISSING), readiness_score. - Persistir el GapReport en JSON. - Especificacion detallada en [gap-engine-mvp.md](#).

4. Metricas globales (dashboard admin) - Tasa de completitud del path por cohorte. - Tiempo promedio hasta dominio del 80%. - Tasa de desercion. - Distribucion de readiness final. - Tasa de insercion laboral (largo plazo). - Para MVP solo lo ve el admin. La empresa no accede a metricas.

Sub-modulos

```
app/modules/  
  students/  
  companies/  
  gap_analysis/  
  global_metrics/
```

Endpoints principales

```
POST   /students                (desde webhook form Google)  
GET    /students/{id}  
POST   /companies  
GET    /companies  
POST   /companies/{id}/positions  
POST   /api/v1/gap-analyses  
GET    /api/v1/gap-analyses/{id}  
GET    /admin/metrics/global
```

Output al Equipo 2

GapReport JSON con el formato definido en `gap-engine-mvp.md`.

Input desde el Equipo 2

Lectura de tablas compartidas (`attempts` , `sessions` , `events`) para sus agregados.

Equipo 2 - Capacitacion + Tutores

Responsabilidades

Parte A - Capacitacion (motor 5P)

1. Generacion del learning path - Consumir el GapReport JSON. - Crear modulos (1 por habilidad faltante), unidades (5P), ejercicios.
2. Contenido adaptativo con IA - Integracion con Google Gemini. - Generar explicaciones, ejercicios y feedback contextualizados al perfil e intereses del estudiante.
3. Ciclo pedagogico 5P - Pasion: anclaje motivacional con la empresa elegida. - Play: sandbox, micro-lecciones de 5 a 10 minutos. - Practica: ejercicios adaptativos, umbral de dominio 80%. - Paciencia y Perseverancia: refuerzo emocional transversal al ciclo.

4. Registro de actividad - Sesiones, intentos, dominio por skill. - Emision de eventos que consume el Equipo 1 para metricas globales.
5. Metricas individuales - Progreso del alumno (vista motivacional). - Datos de seguimiento del tutor (frecuencia, riesgo, etc.).

Parte B - Tutores

6. Padron y asignacion - CRUD de tutores. - Asignar tutor a un estudiante (manual o automatica - a definir).
7. Sistema de alertas internas - Detectar senales (low_perseverance, abandon_risk, lost_passion, anomaly_patience). - Notificar al tutor asignado.
8. Validaciones criticas - Endpoints para que el estudiante suba entregables. - Endpoints para que el tutor valide presentaciones o codigo.
9. Dashboard del tutor - Listado de asignados. - Alertas activas. - Estado de cada estudiante.

Sub-modulos

```
app/modules/  
  learning_paths/  
  learning_content/  
  attempts/  
  sessions/  
  tutors/  
  tutor_assignments/  
  alerts/  
  validations/  
  student_metrics/
```

Endpoints principales

```
POST   /learning-paths  
GET    /learning-paths/{id}  
GET    /learning-paths/{id}/next-unit  
POST   /attempts  
GET    /students/{id}/progress  
GET    /tutors  
POST   /tutors  
POST   /tutor-assignments  
GET    /tutors/{id}/dashboard  
GET    /alerts  
POST   /validations
```

Modulos Compartidos

```
app/modules/  
  users/      (ya implementado)
```

auth/	(post-MVP - login JWT)
shared/	(tipos comunes, db, utilidades)

Pieza	Estado	Quien la mantiene
users/	Implementado	Cualquiera, no se toca mucho
auth/	Post-MVP	Por decidir
students , companies (modelos)	Pendiente	Equipo 1
tutors , learning_paths (modelos)	Pendiente	Equipo 2

Flujo End To End

1. EMPRESA carga oferta (Eq1)
2. ESTUDIANTE completa form Google (Eq1)
3. ESTUDIANTE elige posicion (Eq1)
4. SE GENERA EL GAP REPORT (Eq1)
5. SE GENERA EL LEARNING PATH (Eq2)
6. ESTUDIANTE APRENDE (5P + IA) (Eq2)
7. SISTEMA DETECTA RIESGO (Eq2)
8. TUTOR INTERVIENE SI HACE FALTA (Eq2)
9. ADMIN/EMPRESA VEN METRICAS (Eq1)
10. ESTUDIANTE LISTO PARA POSTULAR

Contratos Entre Equipos

Eq1 -> Eq2: GapReport

Definido en `gap-engine-mvp.md` . Formato JSON estable.

Eq2 -> Eq1: Tablas compartidas

El Equipo 1 puede leer las siguientes tablas para sus metricas agregadas:

- sessions
- attempts
- events (tabla append-only con eventos del flujo pedagogico)
- skill_mastery

Eventos a definir

Pendiente acordar el contrato exacto de eventos que emite el Equipo 2. Lista inicial sugerida:

```
session_started
session_ended
exercise_attempted
```

```
exercise_completed
unit_completed
module_completed
path_completed
mastery_achieved (cuando llega al 80% de una skill)
alert_raised
tutor_intervened
```

Decisiones Pendientes

1. Contrato exacto de eventos Eq2 -> Eq1.
2. Tipo de respuesta del estudiante en ejercicios: texto libre, multiple opcion o codigo.
3. Orden de las fases 5P: fijo por habilidad o decidido por la IA.
4. Algoritmo de asignacion de tutor: manual, round-robin o por especialidad.
5. Responsable del modulo `auth/` cuando se sume.
6. Responsable del frontend.

Resumen

Aspecto	Equipo 1	Equipo 2
Entrada al sistema	Empresa, Estudiante	-
Analisis inicial	Gap Report	-
Motor pedagogico	-	5P + IA Gemini
Acompañamiento humano	-	Tutores + Alertas
Metricas	Globales (admin/empresa)	Individuales (alumno/tutor)
Punto de integracion	Entrega GapReport	Consume GapReport